สรุปและอธิบายเป็นภาษาไทย

เร่งความเร็วการจำลองการซื้อขายเชิงอัลกอริทึมบน GPU กว่า 100 เท่าด้วย Numba

GPU-Accelerate Algorithmic Trading Simulations by over 100x with Numba

Mark J. Bennett (NVIDIA)
NVIDIA Technical Blog, 4 มีนาคม 2025

https://developer.nvidia.com/blog/?p=96652

เกี่ยวกับเอกสารนี้

เอกสารนี้**สรุปและอธิบาย**เนื้อหาของบทความบล็อกข้างต้นเป็นภาษาไทยด้วยถ้อยคำของผู้จัดทำ ไม่ใช่คำแปลคำต่อคำ เพราะบทความต้นฉบับเป็นงานที่มีลิขสิทธิ์ของ NVIDIA ผู้อ่านควรเปิดอ่านต้นฉบับควบคู่กัน โดยเฉพาะรูปภาพและโค้ดในบทความ

นอกจากการสรุป เอกสารนี้ยังเพิ่ม (1) พื้นฐานที่ควรรู้ก่อนอ่าน (2) คำอธิบายและตัวอย่างตัวเลข (3) โค้ดตัวอย่างที่ผู้จัดทำเขียนขึ้นเองและ**ทดสอบรันจริงบน GPU** และ (4) การเชื่อมโยงกับบทความ Avellaneda & Stoikov (2008)

วิธีใช้เอกสารนี้

สาระจากบทความต้นฉบับ (สรุปความ) — ตำแหน่งในบทความ

สรุปใจความของหัวข้อนั้นในบทความต้นฉบับ ด้วยถ้อยคำของผู้จัดทำ

อธิบายแบบง่าย

อธิบายเพิ่มเติมด้วยภาษาง่าย ๆ (ไม่ได้อยู่ในต้นฉบับ)

ตัวอย่าง

ตัวอย่างตัวเลขหรือสถานการณ์ (ไม่ได้อยู่ในต้นฉบับ)

อ่านสมการทีละส่วน

อ่านสมการหรือโค้ดทีละส่วน

สรุปสั้น ๆ

ใจความสำคัญที่ควรจำ

ข้อควรระวัง

ข้อควรระวัง หรือสิ่งที่ต้นฉบับไม่ได้ระบุ

Contents

วิธีใช้เอกสารนี้ **1**

ภาคที่ 0 บทความนี้ในห้านาที **2**

ภาคที่ 1 พื้นฐานที่ควรรู้ก่อนอ่าน **3**

1.1 Backtest, P&L และ Monte Carlo 3

1.2 Alpha คืออะไร 3

1.3 สมุดคำสั่งซื้อขาย 3

1.4 CPU กับ GPU 4

1.5 คำศัพท์ CUDA 4

ภาคที่ 2 อ่านบทความทีละหัวข้อ **5**

2.1 บทนำ 5

2.2 สมุดคำสั่งซื้อขายและโจทย์ 5

2.3 CuPy เทียบกับ Numba 6

2.4 ตัวเลขสุ่ม 7
2.5 Kernel สำหรับการจำลอง 8
2.6 ผลการเร่งความเร็ว 9
2.7 ความเร็วมาจากไหน 10
2.8 บทสรุปของต้นฉบับ 10

ภาคที่ 3 ลงมือทำ: Avellaneda–Stoikov บน GPU **12**
3.1 Kernel 12
3.2 เตรียมข้อมูลและเรียก kernel 12
3.3 ผลลัพธ์จากการรันจริง 13
3.4 แนวทางต่อยอด 14

ภาคที่ 4 อภิธานศัพท์และแหล่งอ้างอิง **15**

ภาคที่ 0 บทความนี้ในห้านาที

สรุปสั้น ๆ — บทความนี้ทำอะไร

1. นักพัฒนาเชิงปริมาณ (quant) ต้องทดสอบกลยุทธ์การซื้อขายย้อนหลัง (backtest) โดย**จำลองตลาดหลาย ๆ ครั้งแบบสุ่ม** (Monte Carlo) เพื่อดูว่ากำไร/ขาดทุนกระจายตัวอย่างไร
2. การจำลองตลาดแบบละเอียดระดับเสี้ยววินาทีตลอดทั้งวัน ทั้งสัปดาห์ หรือทั้งเดือน ใช้เวลานานมากบน CPU
3. ผู้เขียนนำโค้ดจำลองกลยุทธ์ "market making ที่ราคาดีที่สุด พร้อมสัญญาณ alpha ระยะสั้น" จากตำรา *Algorithmic and High-Frequency Trading* มาเขียนใหม่เป็น **CUDA kernel ด้วย Numba**
4. วิธีแบ่งงาน: **หนึ่งเธรดของ GPU = หนึ่งเส้นทางจำลอง** ส่วนการเดินตามเวลายังทำทีละก้าวภายในเธรด
5. ผลลัพธ์บน NVIDIA H200: เร็วกว่า CPU **14 เท่า** (จำลอง 1 วัน) **38 เท่า** (1 สัปดาห์) และ **114 เท่า** (1 เดือน)

อธิบายแบบง่าย — เกี่ยวข้องกับ Avellaneda & Stoikov อย่างไร

บทความ Avellaneda & Stoikov (2008) ที่แปลไว้ก่อนหน้านี้ตอบคำถาม "**ควรตั้งราคาอย่างไร**" และทดสอบด้วยการจำลอง 1000 รอบ ส่วนบทความ NVIDIA นี้ตอบคำถาม "**จะจำลองแบบนั้นให้เร็วขึ้นได้อย่างไร**" ทั้งสองเรื่องจึงต่อกันพอดี: ทฤษฎีให้กลยุทธ์ และ GPU ทำให้ทดสอบกลยุทธ์ได้มากและนานขึ้นในเวลาเท่าเดิม ภาคที่ 3 ของเอกสารนี้แสดงการนำแบบจำลอง Avellaneda–Stoikov มาเขียนเป็น Numba kernel และรันจริง

ภาคที่ 1 พื้นฐานที่ควรรู้ก่อนอ่าน

1.1 Backtest, P&L และการจำลอง Monte Carlo

อธิบายแบบง่าย

- **Backtest** = ทดลองรันกลยุทธ์กับข้อมูลหรือตลาดจำลอง ก่อนนำไปใช้กับเงินจริง
- **P&L (Profit and Loss)** = กำไรหรือขาดทุนที่กลยุทธ์ทำได้
- **Monte Carlo** = แทนที่จะจำลองตลาดครั้งเดียว ให้สุ่มจำลองหลายพันครั้ง แล้วดู**การกระจาย**ของผลลัพธ์ (ค่าเฉลี่ย ส่วนเบี่ยงเบนมาตรฐาน กรณีแย่ที่สุด)

ตัวอย่าง — ทำไมต้องจำลองหลายรอบ

ถ้าจำลองตลาดครั้งเดียวแล้วกลยุทธ์ได้กำไร 70 บาท เราไม่รู้ว่าเป็นโชคหรือฝีมือ แต่ถ้าจำลอง 1000 ครั้งแล้วได้กำไรเฉลี่ย 65 บาท ส่วนเบี่ยงเบนมาตรฐาน 6.6 บาท (ตัวเลขจากตารางที่ 1 ของ Avellaneda & Stoikov) เราจะมั่นใจได้มากขึ้น ความคลาดเคลื่อนของค่าเฉลี่ยลดลงตาม $1/\sqrt{N}$ เมื่อ N คือจำนวนรอบ จำลอง 4 เท่าจึงทำให้ความคลาดเคลื่อนลดลงครึ่งหนึ่ง

1.2 Alpha คืออะไร

อธิบายแบบง่าย

ในวงการลงทุน **alpha** หมายถึงผลตอบแทนส่วนเกินที่ทำได้มากกว่าดัชนีตลาด หลังปรับด้วยอัตราดอกเบี้ยไร้ความเสี่ยงและความเสี่ยงแล้ว ส่วนใน**แบบจำลองของบทความนี้** คำว่า “short-term alpha” หมายถึง**แนวโน้มระยะสั้นของราคา** ที่ตัวแทนคาดการณ์ได้ เช่น ถ้าเพิ่งมีคำสั่งซื้อขนาดใหญ่เข้ามา ราคามักจะขยับขึ้นต่อเล็กน้อยในวินาทีถัดไป แล้วผลนั้นค่อย ๆ จางหายไป

1.3 สมุดคำสั่งซื้อขาย

อธิบายแบบง่าย

สมุดคำสั่งซื้อขาย (limit order book) คือรายการคำสั่งซื้อ (bid) และคำสั่งขาย (ask) ที่รออยู่ในแต่ละระดับราคา **ราคากลาง (midprice)** คือค่าเฉลี่ยของราคาเสนอซื้อดีที่สุดกับราคาเสนอขายดีที่สุด

- **คำสั่งแบบจำกัดราคา (limit order, LO):** รอซื้อ/ขายที่ราคาที่กำหนด
- **คำสั่งตามราคาตลาด (market order, MO):** ซื้อ/ขายทันทีที่ราคาดีที่สุดที่มีอยู่
- “**at the touch**” = วางคำสั่งที่ราคาดีที่สุดพอดี (best bid / best ask)

ดูคำอธิบายพร้อมตัวอย่างตัวเลขเพิ่มเติมได้ในคู่มืออ่าน Avellaneda & Stoikov ภาคที่ 1 หัวข้อ 1.1

1.4 CPU กับ GPU ต่างกันอย่างไร

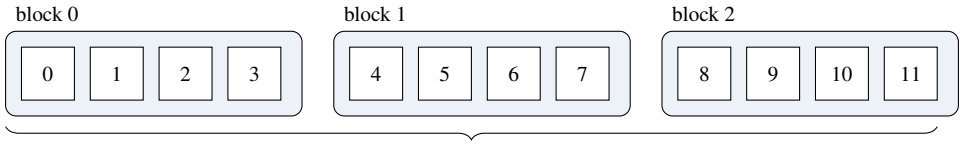
อธิบายแบบง่าย

- **CPU** มีแกนประมวลผลไม่กี่สิบแกน แต่ละแกนเก่งมาก ทำงานซับซ้อนที่ต้องทำต่อเนื่องเป็นลำดับได้ดี
- **GPU** มีแกนเล็ก ๆ (CUDA cores) นับหมื่น บทความระบุว่า GPU สถาปัตยกรรม Hopper และ Blackwell มีมากกว่า 14,000 แกน เหมาะกับงานที่**แบ่งเป็นชิ้นอิสระจำนวนมาก**ซึ่งทำพร้อมกันได้

เปรียบเทียบ: ถ้าต้องตรวจข้อสอบ 1000 ชุด CPU เหมือนอาจารย์ผู้เชี่ยวชาญไม่กี่คน ส่วน GPU เหมือนผู้ช่วยหลายพันคนที่ตรวจคนละชุดพร้อมกัน แต่ถ้างานเป็น “ข้อ 2 ต้องใช้คำตอบข้อ 1” ผู้ช่วยหลายพันคนก็ช่วยอะไรไม่ได้ เพราะต้องรอกันเป็นลำดับ

1.5 คำศัพท์ CUDA ที่ใช้ในบทความ

คำศัพท์	ความหมาย
kernel	ฟังก์ชันที่รันบน GPU โดยหลายเธรดพร้อมกัน
thread (เธรด)	หน่วยงานย่อยที่สุด แต่ละเธรดรันโค้ด kernel เดียวกันกับข้อมูลคนละชุด
block (บล็อก)	กลุ่มของเธรด (เช่น 128 หรือ 256 เธรด)
grid (กริด)	กลุ่มของบล็อกทั้งหมดในการเรียก kernel หนึ่งครั้ง
host / device	host = CPU และหน่วยความจำหลัก, device = GPU และหน่วยความจำของ GPU
`cuda.grid(1)`	หมายเลขประจำตัวของเธรดในกริดแบบหนึ่งมิติ
`cuda.to_device`	คัดลอกข้อมูลจาก host ไป device
`copy_to_host`	คัดลอกข้อมูลจาก device กลับมา host
JIT (just-in-time)	คอมไพล์โค้ด Python เป็นโค้ดเครื่องตอนเรียกใช้ครั้งแรก



grid: หมายเลขเธรด j = `cuda.grid(1)` = 0, 1, 2, …
เธรดที่ j จำลองเส้นทางที่ j

ภาคที่ 2 อ่านบทความทีละหัวข้อ

2.1 บทนำ

สาระจากบทความต้นฉบับ (สรุปความ) — ส่วนเปิดของบทความ

- นักพัฒนาเชิงปริมาณใช้ backtest เพื่อดูว่าอัลกอริทึมซื้อขายทำกำไรหรือขาดทุนอย่างไร และใช้วิธี Monte Carlo เพื่อเห็นช่วงของเส้นทาง P&L ที่เป็นไปได้ GPU ช่วยลดเวลาของงานนี้
- ผู้เขียนเล่าย้อนไปถึงแบบจำลอง Black–Scholes (1973) ว่าเป็นตัวอย่างแรก ๆ ของการใช้คณิตศาสตร์สร้างแบบจำลองตลาด และชี้ว่าตลาดยุคใหม่ต้องใช้แบบจำลองโครงสร้างจุลภาคเฉพาะทาง
- กลุ่มผู้ที่ได้ประโยชน์มีสามกลุ่ม: ผู้ซื้อขาย ตลาดหลักทรัพย์ และผู้บริหารความเสี่ยง ในตลาดปัจจุบันธุรกรรมถูกบันทึกเวลาละเอียดถึงระดับนาโนวินาที ซึ่งสำคัญมากสำหรับผู้ซื้อขายความถี่สูง
- การจำลองอ้างอิงตำรา *Algorithmic and High-Frequency Trading* (Cambridge) และตัวอย่างโค้ดสาธารณะชื่อ "Market Making at the Touch with Short-Term Alpha" ผู้เขียนแนะนำบทความ "Buy Low, Sell High: A High Frequency Trading Perspective" เป็นการอ่านเพิ่มเติม
- ข้อค้นพบหลัก: **ยิ่งจำลองช่วงเวลายาว GPU ยิ่งเร่งความเร็วได้มาก** และผู้ใช้งานต้องตัดสินใจเองว่าต้องจำลองนานเท่าใดจึงมั่นใจพอจะใช้กลยุทธ์จริง
- GPU ที่รองรับและกล่าวถึงในบทความ ได้แก่ H100 NVL/SXM, H200 NVL/SXM, GH200, B200 และ GB200

อธิบายแบบง่าย

ใจความคือ "การทดสอบกลยุทธ์ด้วยการจำลองแบบสุ่มเป็นงานหนัก และ GPU ทำให้เร็วขึ้นมาก" ผู้เขียนไม่ได้คิดกลยุทธ์ใหม่ แต่นำกลยุทธ์จากตำรามาตรฐานมา**เร่งความเร็ว**

ตำรา *Algorithmic and High-Frequency Trading* เขียนโดย Cartea, Jaimungal และ Penalva (2015) ซึ่งต่อยอดแนวคิดของ Avellaneda & Stoikov โดยตรง

2.2 สมุดคำสั่งซื้อขายและโจทย์ที่ต้องแก้

สาระจากบทความต้นฉบับ (สรุปความ) — หัวข้อ "The order book and the problem to solve"

- สถานะของสมุดคำสั่งซื้อขายประกอบด้วยระดับราคาเสนอซื้อ ระดับราคาเสนอขาย และราคากลาง แบบจำลองในบทความใช้ฝั่งละ 10 ระดับ
- มีคำสั่งสองประเภท: คำสั่งแบบจำกัดราคา (LO) และคำสั่งตามราคาตลาด (MO)
- **รูปที่ 1 ในต้นฉบับ:** สมุดคำสั่งของหุ้นเทคโนโลยีขนาดใหญ่ตัวหนึ่ง ช่วงเวลา 34,200–34,220 วินาที (เส้นแดง 10 เส้นคือระดับ ask, เส้นน้ำเงิน 10 เส้นคือระดับ bid, เส้นเขียวคือราคากลาง) ราคากลางลดลงราว 1 ดอลลาร์ (ประมาณ 579 เป็น 578) และมีการแกว่งรุนแรงที่ราว 34,212 วินาที
- **รูปที่ 2 ในต้นฉบับ:** ภาพเดียวกันตลอดทั้งวันทำการ (34,200–57,600 วินาที) ราคากลางลดลงราว 14 ดอลลาร์ และช่วงห่างของสมุดคำสั่งแคบลงหลังจากช่วงเปิดตลาด

ตัวอย่าง — ถอดรหัสตัวเลขเวลา

บทความนับเวลาเป็น “วินาทีนับจากเที่ยงคืน”

- $34{,}200 = 9.5 \times 3600$ วินาที = **9:30 น.** (เวลาเปิดตลาดหุ้นสหรัฐฯ ตามเวลา ET)
- $57{,}600 = 16 \times 3600$ วินาที = **16:00 น.** (เวลาปิดตลาด)
- หนึ่งวันทำการ $= 57{,}600 - 34{,}200 = 23{,}400$ วินาที = 6.5 ชั่วโมง

ตัวเลข 23,400 วินาทีนี้คือหน่วย “1 วัน” ในผลการทดลองความเร็วของหัวข้อ 2.6

อธิบายแบบง่าย — ทำไมช่วงเปิดตลาดจึงผันผวน

ช่วงเปิดตลาดมีข่าวสะสมข้ามคืนและคำสั่งค้างจำนวนมาก ราคาจึงกระโดดแรงและส่วนต่างราคากว้าง เมื่อเวลาผ่านไปตลาดค่อย ๆ ลงตัว ผู้ดูแลสภาพคล่องแข่งกันวางคำสั่ง ส่วนต่างราคาจึงแคบลง ตรงกับสิ่งที่เห็นในรูปที่ 2 ของต้นฉบับ

2.3 CUDA Python: CuPy เทียบกับ Numba

สาระจากบทความต้นฉบับ (สรุปความ) — หัวข้อ “Accelerating simulations with CUDA Python”

- **CuPy** ใช้แทน NumPy ได้เกือบทันที เหมาะกับการคำนวณกับอาร์เรย์ทั้งก้อนนอกฟังก์ชัน
- **Numba** ให้นักพัฒนาเขียน CUDA kernel เองอย่างชัดเจน ควบคุมได้ว่าเธรดไหนทำอะไร
- ผู้เขียนแสดงตัวอย่างการคูณอาร์เรย์สองตัวแบบสมาชิกต่อสมาชิก ทั้งแบบ CuPy (บรรทัดเดียว) และแบบ Numba (kernel ที่แต่ละเธรดคำนวณหนึ่งตำแหน่ง พร้อมตรวจขอบเขตของอาร์เรย์)
- เหตุผลที่เลือก Numba: (1) ควบคุมการทำงานแบบขนานได้ชัดเจน คุ้นเคยสำหรับคนที่เขียน CUDA C++ (2) อาร์เรย์ของ CuPy ใช้ภายใน kernel ของ Numba โดยตรงไม่ได้ และ (3) โค้ดจำลองเดิมเป็นฟังก์ชัน Python ที่วนลูปตามเวลา ซึ่งต้องแปลงเป็น kernel อยู่แล้ว

อ่านสมการทีละส่วน — ตัวอย่างโค้ดที่ผู้จัดทำเขียนเอง: บวกเวกเตอร์สองวิธี

วิธี CuPy เขียนเหมือน NumPy ทุกอย่าง แต่รันบน GPU

```
import cupy as cp
a = cp.arange(1_000_000, dtype=cp.float32)
b = cp.ones_like(a)
c = a + b # runs on the GPU, no kernel written by hand
```

วิธี Numba เขียน kernel เอง ระบุว่าเธรดหนึ่งทำงานกับสมาชิกตัวไหน

```
from numba import cuda
import numpy as np

@cuda.jit
def add_kernel(a, b, c):
    i = cuda.grid(1) # global thread index
    if i < a.size: # guard: the grid may be larger than the data
        c[i] = a[i] + b[i]
```

```
n = 1_000_000
a = cuda.to_device(np.arange(n, dtype=np.float32))
b = cuda.to_device(np.ones(n, dtype=np.float32))
c = cuda.device_array(n, dtype=np.float32)
threads = 256
blocks = (n + threads - 1) // threads # round up
add_kernel[blocks, threads](a, b, c)
```

- `@cuda.jit` บอกว่าฟังก์ชันนี้เป็น kernel
- `[blocks, threads]` คือการกำหนดจำนวนบล็อกและเธรดต่อบล็อกตอนเรียก kernel
- `(n + threads - 1) // threads` คือการปัดขึ้น เพื่อให้มีเธรดพอสำหรับทุกสมาชิก ส่วนเธรดที่เกินมาจะถูกกันด้วย `if i < a.size`

อธิบายแบบง่าย — เลือกใช้อะไรเมื่อไร

	CuPy	Numba
ลักษณะงาน	คำนวณกับอาร์เรย์ทั้งก้อน	ตรรกะซับซ้อน มีลูป มีเงื่อนไข
ความง่าย	ง่ายมาก แก้โค้ด NumPy นิดเดียว	ต้องคิดเรื่องเธรด บล็อก กริด
การควบคุม	น้อย	มาก
งานในบทความนี้	ไม่ เหมาะ (มี ลูป เวลา และ เงื่อนไข ต่อ เส้นทาง)	เหมาะ

ข้อควรระวัง

ตัวอย่างการเรียก kernel ในต้นฉบับมีชื่อฟังก์ชันพิมพ์ผิด (`mullt_kernel`) ถ้าคัดลอกโค้ดจากต้นฉบับไปใช้ ให้แก้ชื่อให้ตรงกับที่ประกาศไว้

2.4 การสร้างตัวเลขสุ่มแบบปกติและแบบสม่ำเสมอ

สาระจากบทความต้นฉบับ (สรุปความ) — หัวข้อ “Normal and uniform variates”

- การจำลองเชิงความน่าจะเป็นต้องใช้ตัวเลขสุ่มแบบปกติมาตรฐาน $N(0,1)$ และแบบสม่ำเสมอ $U(0,1)$
- ผู้เขียนสร้างตัวเลขสุ่ม**ล่วงหน้าทั้งชุด**ด้วย NumPy โดยกำหนด seed เป็น 20 เพื่อให้ผลลัพธ์ทำซ้ำได้ แล้วคัดลอกทั้งชุดไปยัง GPU ด้วย `cuda.to_device` ครั้งเดียว แทนที่จะสร้างทีละตัว
- เก็บสำเนาบน host และ device ให้เหมือนกัน แล้วใช้ `assert` ร่วมกับ `copy_to_host` ตรวจว่าค่าตัวอย่างบน GPU ตรงกับบน CPU

อธิบายแบบง่าย

- **ทำไมสร้างล่วงหน้า:** การคัดลอกข้อมูลระหว่าง CPU กับ GPU มีต้นทุนคงที่ต่อครั้งสูง คัดลอกก้อนใหญ่ครั้งเดียวจึงคุ้มกว่าคัดลอกทีละน้อยหลายครั้ง
- **ทำไมใช้ seed:** เพื่อให้การจำลองบน CPU และ GPU ใช้ตัวเลขสุ่มชุดเดียวกัน ผลลัพธ์จึงเทียบกันได้ตรง ๆ และตรวจความถูกต้องได้
- $N(0,1)$ ใช้เป็น “แรงกระแทก” แบบสุ่มของราคาและ alpha ส่วน $U(0,1)$ ใช้ตัดสินว่าเหตุการณ์จะเกิดหรือไม่

เช่น “ถ้า $U < \lambda\,\mathrm{d}t$ แสดงว่ามีคำสั่งตามราคาตลาดเข้ามา”

ตัวอย่าง — หน่วยความจำที่ต้องใช้ (ผู้จัดทำคำนวณเพิ่ม)

ต้นฉบับไม่ได้ระบุขนาดก้าวเวลาที่ใช้ในการวัดความเร็ว ถ้าสมมุติ $\mathrm{d}t = 0.2$ วินาทีเหมือนตัวอย่าง 30 วินาทีในบทความ

- 1 วัน: $23{,}400/0.2 = 117{,}000$ ก้าว
- อาร์เรย์ตัวเลขสุ่ม 1 ชุด ขนาด 1000 เส้นทาง $\times$ 117,000 ก้าว $\times$ 8 ไบต์ ≈ 0.94 GB
- 21 วัน: ประมาณ 19.7 GB ต่ออาร์เรย์ และต้องใช้หลายอาร์เรย์

ด้วยเหตุนี้การจำลองช่วงยาวจึงต้องใช้ GPU หน่วยความจำมาก เช่น H200 ที่มี 141 GB หรือต้องแบ่งสร้างตัวเลขสุ่มเป็นช่วง ๆ หรือสร้างบน GPU โดยตรง

2.5 Kernel สำหรับการจำลอง

สาระจากบทความต้นฉบับ (สรุปความ) — หัวข้อ “Generate simulation kernel”

- งานหลักของการเร่งความเร็วคือการเขียนฟังก์ชันจำลองที่วนลูปตามเวลาใหม่ให้เป็น Numba kernel
- แต่ละเธรดรับผิดชอบ**หนึ่งเส้นทางจำลอง** (ดัชนี j จาก `cuda.grid(1)` พร้อมตรวจว่า j ไม่เกินจำนวนเส้นทาง) และวนลูปเวลาภายในเธรดนั้น
- สถานะเก็บในอาร์เรย์สองมิติ (เส้นทาง $\times$ เวลา): ราคา, ตัวบ่งชี้ว่ามีคำสั่งตามราคาตลาดหรือไม่, ทิศทางซื้อ/ขายของคำสั่งนั้น, alpha, สถานะคงคลัง และความมั่งคั่ง
- กฎการอัปเดต: ราคากลางเปลี่ยนตาม alpha คูณ $\mathrm{d}t$ บวกแรงกระแทกสุ่ม ส่วน alpha ลดลงแบบเอกซ์โพเนนเชียลเข้าหาศูนย์ บวกแรงกระแทกสุ่ม และกระโดดเมื่อมีคำสั่งตามราคาตลาดเข้ามา
- **รูปที่ 3 ในต้นฉบับ:** alpha ระยะสั้นของหนึ่งใน 1000 เส้นทางจำลอง ($T = 30$ วินาที, $\mathrm{d}t = 0.2$ วินาที) ค่าอยู่ในช่วงประมาณ 0 ถึง 0.006 ดอลลาร์ และส่วนใหญ่เป็นบวก
- **รูปที่ 4 ในต้นฉบับ:** ราคากลางของหนึ่งเส้นทาง พร้อมราคา bid และ ask ที่ห่างออกไปข้างละหนึ่ง tick

อ่านสมการทีละส่วน — แบบจำลอง (เขียนตามแนวของตำรา Cartea–Jaimungal–Penalva)

ต้นฉบับแสดงเฉพาะบางส่วนของสมการ ผู้จัดทำเขียนรูปทั่วไปของแบบจำลองนี้ไว้ดังนี้

ราคากลาง: มีแนวโน้มเท่ากับ alpha และมีความผันผวน σ

$$\mathrm{d}S_t = \alpha_t\,\mathrm{d}t + \sigma\,\mathrm{d}W_t$$

Alpha: ถูกดึงกลับเข้าหาศูนย์ด้วยอัตรา ζ, มีความสุ่ม η, และกระโดดขนาด ε เมื่อมีคำสั่งซื้อตามราคาตลาด ($\mathrm{d}M^+$) หรือคำสั่งขายตามราคาตลาด ($\mathrm{d}M^-$)

$$\mathrm{d}\alpha_t = -\zeta\,\alpha_t\,\mathrm{d}t + \eta\,\mathrm{d}B_t + \varepsilon\,(\mathrm{d}M_t^+ - \mathrm{d}M_t^-)$$

เมื่อแปลงเป็นก้าวเวลา $\mathrm{d}t$ (สอดคล้องกับคำอธิบายในต้นฉบับ)

$$\alpha_{i+1} = \mathrm{e}^{-\zeta\,\mathrm{d}t}\,\alpha_i + \eta\sqrt{\mathrm{d}t}\;Z_i' + \varepsilon\cdot(+1 \text{ ถ้ามี MO ซื้อ},\ -1 \text{ ถ้ามี MO ขาย},\ 0 \text{ ถ้าไม่มี})$$

- $\mathrm{e}^{-\zeta\,\mathrm{d}t}$ คือการลดลงแบบเอกซ์โพเนนเชียล ถ้าไม่มีอะไรมากระทบ alpha จะค่อย ๆ จางเข้าหาศูนย์

- $\sqrt{dt}$ มาจากคุณสมบัติของการเดินสุ่ม (ความแปรปรวนโตตามเวลา ส่วนเบี่ยงเบนมาตรฐานโตตามรากที่สองของเวลา)
- พจน์ ε ทำให้ alpha **ขึ้น**เมื่อมีคนรีบซื้อ และ**ลง**เมื่อมีคนรีบขาย ซึ่งสะท้อนว่าคำสั่งตามราคาตลาดมักบอกทิศทางราคาระยะสั้น

ตัวอย่าง — alpha จางหายเร็วแค่ไหน

ช่วงเวลาที่ alpha ลดเหลือครึ่งหนึ่ง (half-life) คือ $\ln 2/\zeta$ ถ้า $\zeta = 2$ ต่อวินาที (ค่าสมมุติ ต้นฉบับไม่ได้ระบุค่าพารามิเตอร์) half-life $= 0.35$ วินาที แปลว่าข้อมูลจากคำสั่งซื้อขนาดใหญ่มีผลต่อราคาเพียงเสี้ยววินาที นี่คือเหตุผลที่ต้องจำลองละเอียดระดับเสี้ยววินาที

อธิบายแบบง่าย — ทำไมแบ่งงานแบบ “หนึ่งเธรด = หนึ่งเส้นทาง”



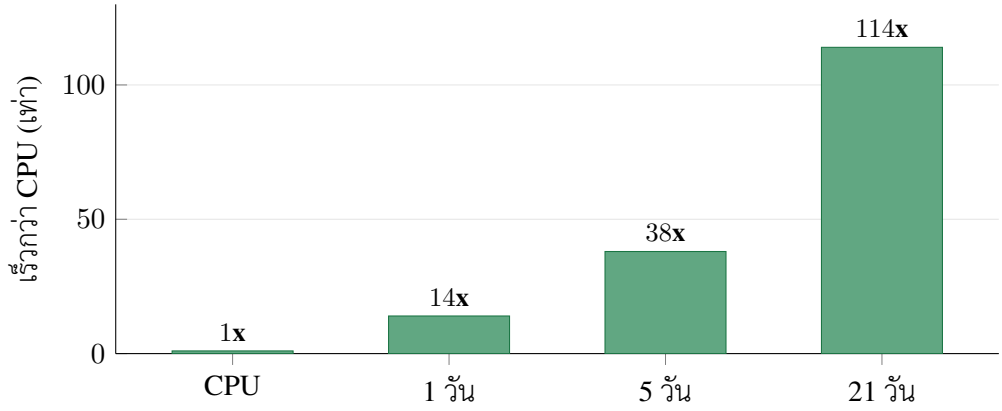
- **แนวเวลา** ทำขนานไม่ได้ เพราะราคา ณ ก้าวที่ $i+1$ ต้องรู้ราคา ณ ก้าวที่ i ก่อน
- **แนวเส้นทาง** ทำขนานได้สมบูรณ์ เพราะแต่ละเส้นทางไม่ขึ้นต่อกันเลย

จึงให้แต่ละเธรดเดินตามเวลาของเส้นทางตัวเองตั้งแต่ต้นจนจบ ภาคที่ 3 มีตัวอย่าง kernel แบบนี้ที่รันได้จริง

2.6 ผลการเร่งความเร็ว

สาระจากบทความต้นฉบับ (สรุปความ) — หัวข้อ “Acceleration results”

ผู้เขียนเทียบ GPU NVIDIA H200 SXM5 กับ CPU โดยจำลอง 1000 เส้นทาง ซึ่งพอสำหรับคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานของราคากลาง alpha และ P&L ผลคือความเร็วที่เพิ่มขึ้นมากขึ้นตามความยาวของช่วงเวลาที่จำลอง (ตัวเลขตามรูปที่ 5 ของต้นฉบับ แสดงซ้ำในแผนภูมิด้านล่าง)



ข้อมูลจากรูปที่ 5 ของบทความต้นฉบับ (H200 SXM5 เทียบกับ CPU, 1000 เส้นทาง) วาดใหม่โดยผู้จัดทำ
1 วัน = 23,400 วินาที, 5 วัน = 1 สัปดาห์ทำการ, 21 วัน = 1 เดือนทำการ

ข้อควรระวัง — สิ่งที่ต้นฉบับไม่ได้ระบุ

บทความไม่ได้บอกรุ่นของ CPU ที่ใช้เทียบ ไม่ได้บอกว่าโค้ด CPU ใช้หลายแกนหรือไม่ ไม่ได้บอกขนาดก้าวเวลา และไม่ได้บอกว่าการจับเวลารวมการคัดลอกข้อมูลและการคอมไพล์ JIT หรือไม่ ดังนั้นตัวเลข 14–114 เท่า**เทียบข้ามระบบอื่นตรง ๆ ไม่ได้** ควรวัดใหม่บนเครื่องของตัวเองเสมอ

2.7 ความเร็วมาจากไหน

สาระจากบทความต้นฉบับ (สรุปความ) — หัวข้อ "The source of the speedup"

- ปัญหามีสองมิติ: **มิติเวลา**ทำเป็นลำดับ เพราะราคาแต่ละก้าวขึ้นกับก้าวก่อน GPU จึงไม่ได้เปรียบในมิตินี้ ส่วน**มิติเส้นทาง**แต่ละเส้นทางเป็นอิสระต่อกัน ความเร็วจึงมาจากมิตินี้
- ยิ่งจำลองหลายเส้นทาง การประมาณการแจกแจงของผลลัพธ์ยิ่งแม่นยำ เมื่อใช้ตั้งแต่ 1000 เส้นทางขึ้นไป โดยเฉพาะกับช่วงเวลายาว ปริมาณงานขนานของ GPU จะได้เปรียบชัดเจน
- ผู้เขียนอ้างถึงงานของ NVIDIA ด้านการเรียนรู้ของเครื่องกับสมุดคำสั่งซื้อขายเพื่อทำนายราคา (2022 และ 2023) และงานจำแนกประเภทปี 2018 ที่เป็นพื้นฐาน

อธิบายแบบง่าย — คำอธิบายเพิ่มเติมจากผู้จัดทำ: ทำไมจำลองนานขึ้นแล้วเร็วขึ้นมากขึ้น

เวลาที่ใช้บน GPU แยกได้สองส่วน

$$\text{เวลา GPU} \approx \underbrace{\text{ต้นทุนคงที่}}_{\text{เริ่ม kernel, คัดลอกข้อมูล, คอมไพล์}} + \underbrace{\text{งานต่อก้าว} \times \text{จำนวนก้าว}}_{\text{ทำขนาน}}$$

เมื่อจำลองสั้น ต้นทุนคงที่กินสัดส่วนใหญ่ ความเร็วที่ได้จึงน้อย เมื่อจำลองนานขึ้น ต้นทุนคงที่ถูกเฉลี่ยออกไป สัดส่วนของงานที่ทำขนานได้เพิ่มขึ้น ความเร็วที่ได้จึงสูงขึ้น นี่เป็นคำอธิบายเชิงคุณภาพของผู้จัดทำ ต้นฉบับไม่ได้แยกเวลาให้ดู

อีกข้อสังเกต: 1000 เส้นทางใช้แค่ 1000 เธรด ขณะที่ GPU มีแกนมากกว่า 14,000 แกน GPU จึงยังทำงานไม่เต็มกำลัง ถ้าเพิ่มจำนวนเส้นทาง ความเร็วที่ได้มักจะเพิ่มขึ้นอีก (ดูผลทดลองในภาคที่ 3)

2.8 บทสรุปของต้นฉบับ

สาระจากบทความต้นฉบับ (สรุปความ) — หัวข้อ "Conclusion"

- ผู้เขียนระบุว่าแนวปฏิบัติปัจจุบันในการจำลองคำสั่งซื้อขายความถี่สูงอิงตำรา *Stochastic Calculus for Finance II* และแนะนำ *Financial Analytics with R* สำหรับเรื่องความเหมาะสมของการแจกแจงปกติและผลตอบแทนแบบลอการิทึมกับพฤติกรรมตลาด ส่วนทฤษฎีการหาค่าเหมาะสมที่สุด (dynamic programming) ดูได้ใน *Algorithmic and High-Frequency Trading*
- การจำลองละเอียดระดับเสี้ยววินาทีตลอดวันทำการเป็นงานหนักแม้บน CPU หลายแกน ถ้าจะครอบคลุมหลักทรัพย์จำนวนมากในหลายตลาดต้องใช้ GPU หลายตัว
- เครื่องมือที่แนะนำ: **RAPIDS** สำหรับงานวิทยาการข้อมูลบน GPU (ติดตั้ง NVIDIA driver และ CUDA toolkit ก่อน) และ **Numba** สำหรับเขียน CUDA kernel ด้วย Python โดย CuPy ง่ายกว่าสำหรับการแปลงอาร์เรย์ทั่วไป แต่ Numba ควบคุมมิติการทำงานขนานได้ดีกว่า จึงเหมาะกับงานนี้

- ผู้เขียนแนะนำเซสชัน S72692 “AI for Safe and Efficient Trading in Electronic Markets” ในงาน NVIDIA GTC 2025

สรุปสั้น ๆ — บทเรียนสำหรับงาน HPC

1. หา**มิติที่เป็นอิสระ** ของปัญหา (ในที่นี้คือเส้นทางจำลอง) แล้วกระจายมิตินั้นไปยังเธรด
2. มิติที่ต้องทำเป็นลำดับ (เวลา) ให้คงเป็นลูปภายในเธรด
3. ลดการคัดลอกข้อมูลระหว่าง CPU กับ GPU โดยเตรียมข้อมูลเป็นก้อนใหญ่ครั้งเดียว
4. ใช้ seed และเทียบผล GPU กับ CPU เพื่อตรวจความถูกต้องก่อนวัดความเร็ว
5. ความเร็วที่ได้ขึ้นกับขนาดปัญหา ต้องวัดหลายขนาด

ภาคที่ 3 ลงมือทำ: Avellaneda–Stoikov บน GPU ด้วย Numba

ภาคนี้ผู้จัดทำเขียนขึ้นเองทั้งหมด เพื่อแสดงแนวคิดของบทความ NVIDIA กับแบบจำลองที่ง่ายกว่าและคุ้นเคยแล้ว คือ การจำลองใน §3.3 ของ Avellaneda & Stoikov (2008) โค้ดนี้**ทดสอบรันจริงแล้ว**บน NVIDIA GeForce RTX 5060 Ti (Numba 0.68 + numba-cuda)

3.1 Kernel

```python
import math
import numpy as np
from numba import cuda

@cuda.jit
def as_kernel(Z, Ua, Ub, gamma, sigma, k, A, T, dt, s0, pnl, q_final):
    j = cuda.grid(1) # one thread = one simulation path
    if j >= Z.shape[0]:
        return
    s, q, x = s0, 0.0, 0.0
    liq = (2.0 / gamma) * math.log(1.0 + gamma / k)
    for i in range(Z.shape[1]): # the time loop stays sequential
        tau = T - i * dt
        r = s - q * gamma * sigma * sigma * tau # eq. (29)
        spread = gamma * sigma * sigma * tau + liq # eq. (30)
        da = r + 0.5 * spread - s # ask distance
        db = s - (r - 0.5 * spread) # bid distance
        if Ua[j, i] < A * math.exp(-k * da) * dt: # our ask is lifted
            q -= 1.0
            x += s + da
        if Ub[j, i] < A * math.exp(-k * db) * dt: # our bid is hit
            q += 1.0
            x -= s - db
        s += sigma * math.sqrt(dt) * Z[j, i] # random mid-price step
    pnl[j] = x + q * s
    q_final[j] = q
```

อ่านสมการทีละส่วน — อ่านโค้ดทีละส่วน

- `j = cuda.grid(1)`: เธรดนี้รับผิดชอบเส้นทางที่ j (หลักการเดียวกับบทความ NVIDIA)
- ลูป `for i`: เดินตามเวลา 200 ก้าว ($T/\mathrm{d}t = 1/0.005$) ภายในเธรด
- `r` และ `spread`: สูตร (29) และ (30) ของ Avellaneda & Stoikov
- `Ua[j,i] < A*exp(-k*da)*dt`: ใช้ตัวเลขสุ่ม $U(0,1)$ ตัดสินว่าคำสั่งขายของเราถูกจับคู่ในก้าวนี้หรือไม่ ด้วยความน่าจะเป็น $\lambda^a\,\mathrm{d}t$
- `s += sigma*sqrt(dt)*Z[j,i]`: ราคากลางเดินสุ่มด้วยตัวเลขสุ่ม $N(0,1)$ (บทความต้นฉบับใช้ $\pm\sigma\sqrt{\mathrm{d}t}$ แบบสองค่า ซึ่งให้ผลใกล้เคียงกัน)

3.2 เตรียมข้อมูลและเรียก kernel

```
n_paths, T, dt = 1000, 1.0, 0.005
n_steps = int(round(T / dt))
rng = np.random.default_rng(20) # fixed seed, as in the blog
Z = cuda.to_device(rng.standard_normal((n_paths, n_steps)))
Ua = cuda.to_device(rng.random((n_paths, n_steps)))
Ub = cuda.to_device(rng.random((n_paths, n_steps)))
pnl = cuda.device_array(n_paths)
q_final = cuda.device_array(n_paths)

threads = 128
blocks = (n_paths + threads - 1) // threads
as_kernel[blocks, threads](Z, Ua, Ub, 0.1, 2.0, 1.5, 140.0,
                           T, dt, 100.0, pnl, q_final)
p = pnl.copy_to_host()
print(p.mean(), p.std())
```

3.3 ผลลัพธ์ที่ได้จากการรันจริง

ความถูกต้อง: เทียบกับตารางที่ 1–3 ของ Avellaneda & Stoikov (2008) (กลยุทธ์สถานะคงคลัง, 1000 เส้นทาง)

	กำไรเฉลี่ย		Std(กำไร)		Std(q สุดท้าย)	
γ	บทความ	GPU ของเรา	บทความ	GPU ของเรา	บทความ	GPU ของเรา
0.1	65.0	64.3	6.6	6.5	2.9	3.0
0.01	68.6	67.9	8.7	8.7	5.1	5.3
1	31.4	31.2	5.0	4.8	1.7	1.6

ค่าใกล้เคียงกับบทความทุกตัว ความต่างเล็กน้อยมาจากการสุ่มคนละชุด นอกจากนี้ผลจาก GPU ยังตรงกับผลจากเวอร์ชัน CPU (Numba `@njit`) ที่ใช้ตัวเลขสุ่มชุดเดียวกันทุกเส้นทาง (ตรวจด้วย `np.allclose`)

ความเร็ว: เทียบ GPU กับ CPU แกนเดียวที่คอมไพล์ด้วย `@njit` (โค้ดเดียวกัน, 200 ก้าวเวลา, ไม่รวมเวลาคอมไพล์และคัดลอกข้อมูล)

จำนวนเส้นทาง	CPU (1 แกน)	GPU RTX 5060 Ti	เร็วขึ้น
1,000	5.5 ms	0.35 ms	15.6 เท่า
100,000	545.7 ms	6.65 ms	82.1 เท่า

อธิบายแบบง่าย — อ่านผลความเร็ว

- เมื่อเพิ่มเส้นทางจาก 1,000 เป็น 100,000 เวลา CPU เพิ่มขึ้นราว 100 เท่าตามจำนวนงาน แต่เวลา GPU เพิ่มขึ้นเพียงราว 19 เท่า เพราะ 1,000 เธรดยังใช้ GPU ไม่เต็มกำลัง
- ผลนี้ยืนยันข้อสังเกตในหัวข้อ 2.7: ยิ่งงานขนานมาก GPU ยิ่งได้เปรียบ
- ตัวเลขเหล่านี้เป็นการวัดบนเครื่องของผู้ใช้ ซึ่งต่างจากเครื่องในบทความ (H200) จึงไม่ควรนำไปเทียบกับ 14–114 เท่าของต้นฉบับโดยตรง และ CPU หลายแกนจะลดช่องว่างลง

ข้อควรระวัง — ข้อสังเกตการติดตั้ง

ระหว่างทดสอบพบว่า numba-cuda เวอร์ชันที่ติดตั้งไม่ทำงานกับ NumPy 2.5 (ฟังก์ชัน `np.row_stack` ถูกถอดออก) ต้องใช้ NumPy เวอร์ชันต่ำกว่า 2.4 แทน

3.4 แนวทางต่อยอดสำหรับโครงงาน

- วัดความเร็วเมื่อเปลี่ยนจำนวนเส้นทาง (1K–1M) และจำนวนก้าวเวลา แล้ววาดกราฟ speedup เหมือนรูปที่ 5 ของต้นฉบับ
- ทดลองขนาดบล็อก (64, 128, 256, 512 เธรด) และดูผลต่อความเร็ว
- สร้างตัวเลขสุ่มบน GPU โดยตรง (เช่น `numba.cuda.random`) แทนการสร้างบน CPU แล้วคัดลอก เพื่อลดหน่วยความจำและเวลาคัดลอก
- ใช้ `float32` แทน `float64` แล้วตรวจว่าผลยังถูกต้องหรือไม่ (GPU ทั่วไปคำนวณ float32 เร็วกว่ามาก)
- เทียบกับ CPU หลายแกน (`numba.prange`) เพื่อให้การเปรียบเทียบยุติธรรมขึ้น
- เพิ่ม alpha ระยะสั้นแบบในบทความ NVIDIA เข้าไปในแบบจำลอง Avellaneda–Stoikov

ภาคที่ 4 อภิธานศัพท์และแหล่งอ้างอิง

ภาษาไทย	English	ความหมายสั้น ๆ
การทดสอบย้อนหลัง	backtest	ทดลองกลยุทธ์กับข้อมูลหรือการจำลอง
การจำลองมอนติคาร์โล	Monte Carlo simulation	สุ่มจำลองหลายครั้งเพื่อดูการแจกแจง
เส้นทางจำลอง	simulation path	ผลการจำลองหนึ่งรอบตลอดช่วงเวลา
alpha ระยะสั้น	short-term alpha	แนวโน้มราคาระยะสั้นที่คาดการณ์ได้
การดึงกลับสู่ค่าเฉลี่ย	mean reversion	ค่าถูกดึงกลับเข้าหาระดับปกติ
ราคาดีที่สุด	the touch (best bid/ask)	ราคาเสนอซื้อสูงสุด / เสนอขายต่ำสุด
ขนาดราคาขั้นต่ำ	tick	ระยะห่างราคาที่เล็กที่สุดที่ตลาดยอมรับ
เคอร์เนล	kernel	ฟังก์ชันที่รันบน GPU
เธรด / บล็อก / กริด	thread / block / grid	หน่วยการทำงานขนานของ CUDA
โฮสต์ / อุปกรณ์	host / device	CPU / GPU
คอมไพล์ขณะรัน	JIT compilation	แปลงโค้ด Python เป็นโค้ดเครื่องตอนเรียกใช้ครั้งแรก
ตัวแปรสุ่มแบบปกติ / สม่ำเสมอ	normal / uniform variate	ตัวเลขสุ่ม $N(0,1)$ / $U(0,1)$
เมล็ดสุ่ม	random seed	ค่าตั้งต้นที่ทำให้สุ่มซ้ำได้เหมือนเดิม
อัตราเร่ง	speedup	เวลา CPU หารด้วยเวลา GPU

แหล่งอ้างอิง

- M. J. Bennett, “GPU-Accelerate Algorithmic Trading Simulations by over 100x with Numba,” NVIDIA Technical Blog, 4 มีนาคม 2025. `https://developer.nvidia.com/blog/?p=96652`
- Á. Cartea, S. Jaimungal, J. Penalva, *Algorithmic and High-Frequency Trading*, Cambridge University Press, 2015.
- M. Avellaneda, S. Stoikov, “High-frequency trading in a limit order book,” *Quantitative Finance* 8(3), 217–224, 2008.
- Numba CUDA documentation: `https://nvidia.github.io/numba-cuda/`